No of additinnal sheets

LOURDES MATHA COLLEGE OF SCIENCE & TECHNOLOGY

LOURDES HILLS. KUTTICHAL P. O., THIRUVANANTHAPURAMM - 695 574

ANSWER BOOK

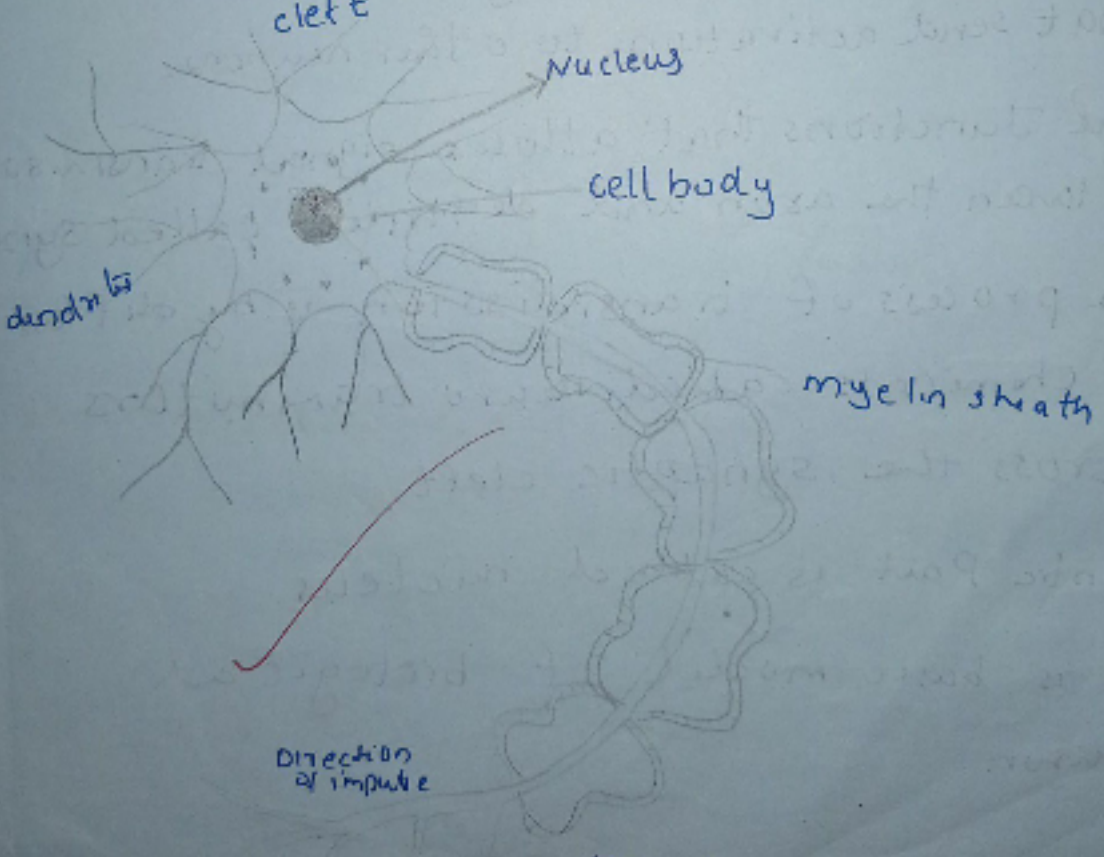
Name. Kavya Chandran GR Class No. 23 Branch. MCA

Subject. Deep Learning Semester S3

Mark Tabulation

Mark Divisions	Q.1	Q.2	Q.3	Q.4	Q.5	Q.6	Q.7	Q.8	Q.9	Q.10	Q.11	Q.12	Q.13	Q.14	Q.15
	3	3	3	3	2½	3	5½	4½							
Total															
Grand Total												28			

6.



The majority of neurons encode their activations or output as a series of brief electrical pulses.

→ The neuron's cell body (soma) processes the incoming activations and converts them into output activation

→ The neuron's nucleus contains the genetic material in the form of DNA. This exists in most types of cells not just neurons.

→ Dendrites are fibres which emanate from the cell body and provide the receptive zones the receive activation from other neurons

→ Axons are fibres acting as transmission lines that send activation to other neurons.

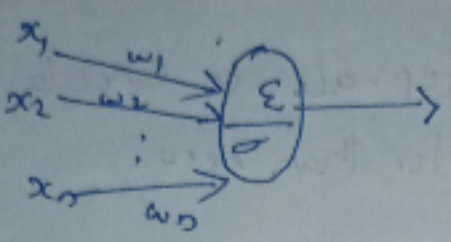
→ The Junctions that allows signal transmission between the axon and dendrites called Synapse

→ The process of transmission is by diffusion of chemicals called neuro transmitters across the synaptic cleft

→ centre Part is called nucleus

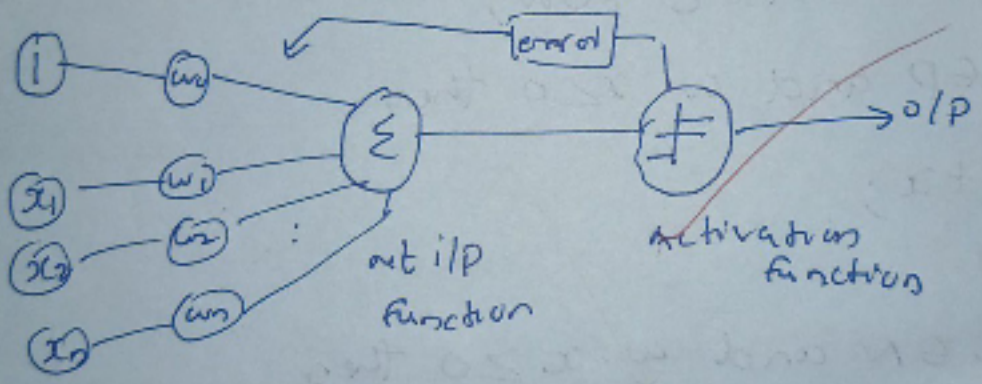
→ It is basic model of biological neuron.

4. perceptron is a building block of an Artifical neural network. Single perceptron can only be used to implement linearly seperable function.



<u>components</u>

input nodes
weight and Bias
Activation functions → sign functions
→ step "
→ sigmoid "



step 1: $\Sigma w_i * x_i = x_1 * w_1 + x_2 * w_2 \ldots w_n * x_n$

$\Sigma w_i * x_i + b$

step 2: $Y = f(\Sigma w_i * x_i + b)$

it is two steps of perceptron.

Perceptron is a machine Language algorithm for supervised learning of binary classifiers. In perception the weight coefficient is automatically learned. The activation function applies a step rule to check whether the weight function greater than zero.

Algorithm

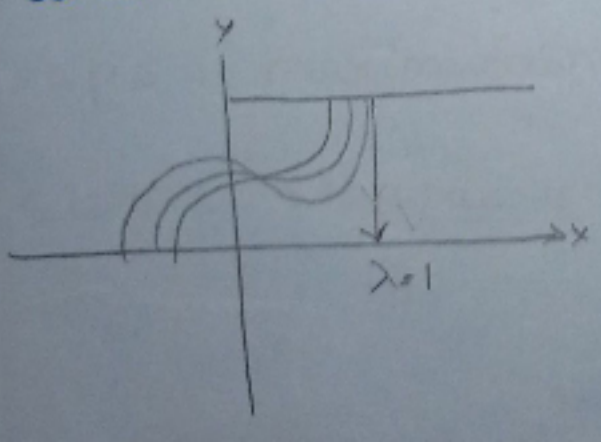
P ← ilps with label 1;
N ← ilps with label 0;
initialize a randomly;
while ! Convergence do
Pick random x ∈ PUN;
if x ∈ P and w.x < 0 then
w = w + x;
end
if x ∈ N and w.x ≥ 0 then
w = w - x;
end
end.

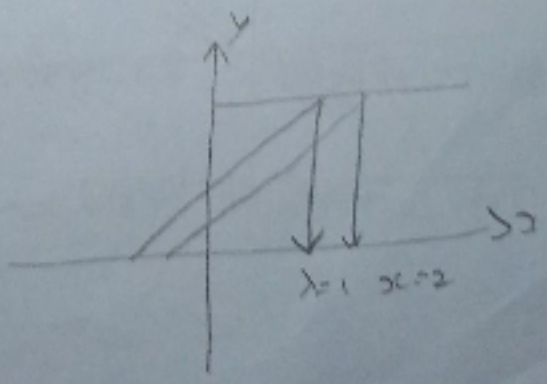
1. Sigmoidal function: widely used in back propation network because of the relationship between the value of the function at a point and value of drivative at that point which reduce the computational burden during training.

Two types:

a) Binary sigmoid function
b) Bipolar sigmoid function

Binary sigmoid function: This activation function performs input editing between 0 and 1. It is positive in nature. It is always bounded which means its output cannot be less than 0 and more than 1. It is also strictly increasing in nature which means more the input higher would be the output.



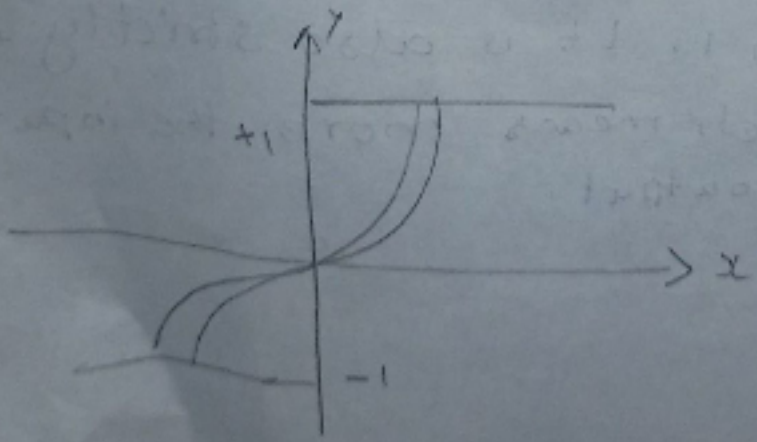


$$f(x) = sigm(x) = \frac{1}{1+\exp(-x)}$$

Bipolar sigmoidal function : This activation function performs input editting between -1 and 1. It can be positive or negative in nature It is always bounded which means its o/p cannot be less than -1 and more than 1. It is also strictly increasing in nature like sigmoid function.

$$F(x) = sigm(x) = \frac{2}{1+\exp(-x} - 1$$

$$= \frac{1-\exp(x)}{1+\exp(x)}$$



2. Gradient desent Algorithm: Iteratively calculate the next point using gradient of the current position. Then scalar it by a learning rate and substracts the obtained values from the current position. 'η' eta learning rate it scales the gradient and thus control the step size when eta become smaller than longer is gradient desent converges.

step 1: choose a starting point (initialization
step 2: calculate gradient at this point
step 3: make a scaled step in the opposite direction to the gradient (minimize)
step 4: Repeat step 2 and step 3 until one of the criteria done
step 5: maximum number of iteration is reached
step 6: step size is smaller than the tolerence

Parameters

Starting point
Gradient function
Learning rate
Maximum no: of iterations
Tolerence (stoping condition)

Algorithm

initialize w, b;
iterate over data;
compute y'
compute $L(w, b)$

$$w_{t+1} = w_t - \eta \Delta w_t$$

$$b_{t+1} = b_t - \eta \Delta b_t$$

where $\Delta w_t = \frac{\partial L(w,b)}{\partial w}$

at $w = w_t, b = b_t$

$$\Delta b_t = \frac{\partial L(w,b)}{\partial b}$$ at $w = w_t$, $b = b_t$

5. TenRank shapes and Types

Tensor flow programs use a tensor data structure to represent all data. A tensor has a static type of and dynamic dimensions

Rank

In the tensor flow system, tensors are described by a unit of dimensionality known as rank.

Tensor rank is the number of dimensions of the tensor

for example the following tensor has a rank 2

t : [[1,2,3], [4,5,6] [7,8,9]]

Shape

The tensor flow documentation uses three notational conventions to describe tensor dimensionality rank shape and dimension number.

Types

for rank 0 shape 1 $S_1 [D_0]$

for Rank 1 shape 2 $S_2 [D_0, D_1]$

for Rank 2 Shape 3 $S_3 [D_0, D_1, D_2]$

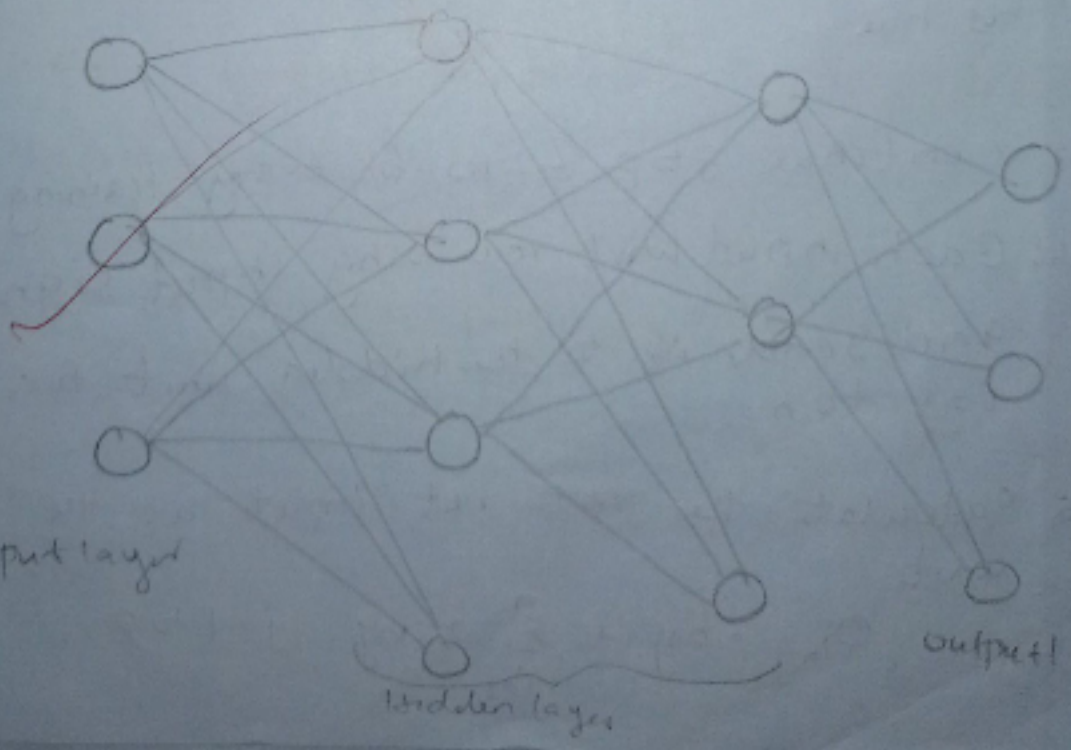
for Rank 3 shape 4 $S_4 = [D_0, D_1, D_2, D_3]$

Types

Tensor consist of different data types on different data classes ~~or~~ function.

Data structure to represent all the data in the tensor types.

1. Back propagation Neural (BPN) is a multiple ~~target~~ layer neural network consisting of the input layer atleast one hidden layer and output layer. As its name suggests back propagation will take place in this network The error which is calculated at the output layer by comparing the target output and the actual output will be propagated back towards the input layer



Three phase

Phase 1 - feed forward

Phase 2 - Back propogation

Phase 3 - updating of weights

Steps

Step 1: Initialize the following to start the training
- weight
- learning rate α

Step 2: Continue step 3-11 when the stopping condition is true

Step 3: Continue step 4-10 for every training pair

Step 4: Each input unit receiving input signal x_i, and sends it to the hidden unit for all $i = 1$ to n.

Step 5: Calculate the ~~rate~~ net input at the hidden unit

$$Q_{inj} = b_{oj} + \sum_{i=1}^{n} x_i v_{ij} \quad j = 1 \text{ to } p$$

step 6: calculate the net input at the output layer unit

step 7: Compute the error correcting term in correspondence with the target pattern received at each output unit.

step 8: now each hidden unit will be in the sum of its delta inputs from the output units

step 9: Each output unit (y_k k = 1 to m) update the weight and bias

step 10: Each output units (Z_j j = 1 to p) update the weight and bias

step 11: check for the stopping condition, which may be either the no: of epoch reached or the target output matches the actual output.

3. o/p = n - f + 1

= 12 - 3 + 1

= 10

Given

n = 12, f = 3, S = 1

o/p = n - f + 1

where, o/p matrix size f, filter size

The obtain matrix according to given convolution is 10.

8. Batch Normalization

In batch normalization in which it is done to access different forms. Batch normalization refers that it is a technique the activates function of the each large which main simply zero centure and normal the input the scaling and satisfy the result using ~~then scaling and satisfy~~ the new parameters Layer

that the operation lets the model learn the optimal scale and means of the input for each input. In order to zero center and normalize the input the algorithm needs to estimate the input to the process in which mean and standard deviation. It does by evaluating the input and standard deviation of inputs over the current batch

Algorithm.

$$\mu_B = \frac{1}{m_B} \sum_{i=1}^{m_B} x(i)$$

$$\sigma_B^2 = \frac{1}{m_B} \sum_{i=1}^{m_B} (x^{(i)} - \mu_B)^2$$

$$x^n(i) = \frac{x^{(i)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$z^{(i)} = \gamma x^n(i) + \beta$$